

클라이언트 프로그래밍 포트폴리오

임채은



010-5744-2840



ce0118ce@naver.com



증명사진 찍고
대체할 예정

임채은



🎓 학력

2019.03 ~ 2022.02

부천여자고등학교 졸업

2022.03 ~ 2027.02

한국공학대학교 게임공학과 졸업 예정

🖋 기술 스택



git





프로젝트 목차

1. Space Robber
2. Magical Subway
3. 대초원의 모험



1. Space Robber



한국공학대학교 졸업작품

▼ 게임 설명

- Unreal Engine5 1인칭 협동 자원 수집 멀티 플레이 게임
- 1~4인 실시간 협동 플레이를 기반으로 한 자원 수집·생존 게임입니다. 플레이어는 제한된 시간 동안 맵을 탐험하며 자원을 수집하고, 몬스터와 함정을 피해 제한 시간 내 기지로 복귀해야 합니다. 수집한 자원을 판매하여 장비를 구입하고, 할당량을 채우는 것을 목표로 합니다.
- 또한 [OpenCV](#) 기반 실시간 감정 인식 시스템을 적용하여 플레이어의 감정 상태에 따라 게임 환경과 난이도가 변화하는 몰입형 게임 플레이를 제공합니다.



영상 찍고
널을 예정



▼ 게임 개발

- 인원: 3명(클라이언트 1명 – 본인, 서버 1명, 기획 1명)
- 기간: 2025.12 ~ 2026.06 (7개월)
- 담당 업무
 - 클라이언트(네트워크 동기화, OpenCV 시스템 구현, 인벤토리 · 상점 · UI, 마이크 입출력 시스템)
 - 목적: 기획 문서를 바탕으로 Unreal Engine 5.6 환경에서 1인칭 3D 온라인 게임을 구현하고, 서버 패킷 처리와 객체 동기화를 통해 실시간 멀티플레이 시스템을 개발하는 것입니다. 더불어 OpenCV 라이브러리를 활용한 감정 인식 기능을 게임에 연동하여 플레이어의 몰입감을 향상시키고자 했습니다.
 - 기술: Unreal Engine 5.6, C++, Winsock(TCP/IP, UDP, Select I/O Model, Multi Thread)



1. Space Robber



게임 구현 설명

▼ 네트워크 동기화

- GamelInstance에서 네트워크 송수신 모듈을 (GameNetwork) 관리하고, 게임 로직이 실행되는 Main Thread와 별도로 Recv Thread를 생성하여 네트워크 처리를 수행했습니다.

▼ 플레이어 이동 동기화

- 플레이어의 위치 동기화는 MyPlayer::Tick()에서 수행하지만, 매 프레임 패킷을 전송하지 않도록 최적화했습니다.
- 일정 시간(MOVE_PACKET_SEND_DELAY)마다 전송 여부를 검사하고, 현재 위치와 회전을 이전에 전송한 값과 비교합니다. 위치와 회전에 실제 변화가 발생한 경우에만 패킷을 전송하도록 구현했습니다.
- 이를 통해 불필요한 패킷 처리를 줄여 서버 부하와 네트워크 트래픽을 감소시켰습니다.

- GamelInstance에서 GameNetwork와 스레드 생성

```
void UMain::ConnectServer()
{
    _gameNetwork = new GameNetwork();

    _recvRunnable = new NetworkRunnable(this);
    _recvThread = FRunnableThread::Create(_recvRunnable, TEXT("RecvThread"));
}
```

- MyPlayer에서 일정 주기마다 위치 · 회전 패킷 전송 처리

```
void AMyPlayer::Tick(float DeltaTime)
{
    Super::Tick(DeltaTime);

    _movePacketSendTimer -= DeltaTime;
    _interactionTimer -= DeltaTime;

    // MovePacket Send
    if (_movePacketSendTimer <= 0.f)
    {
        FVector currentLocation = GetActorLocation();
        FRotator currentRotation = GetActorRotation();

        // 이전 전송 값 비교
        if (!currentLocation.Equals(_lastSentLocation, 0.01f) || !currentRotation.Equals(_lastSentRotation, 0.01f))
        {
            // 이동이나 회전이 감지되었을 때만 타이머 초기화하고 패킷 전송함
            _movePacketSendTimer = MOVE_PACKET_SEND_DELAY;

            _lastSentLocation = currentLocation;
            _lastSentRotation = currentRotation;

            if (UMain* GameInstance = Cast<UMain>(GetGameInstance()))
                GameInstance->SendLocalPosition();
        }
        else
        {
            // 움직임이 없다면 타이머 0으로 고정(움직였을 때 바로 패킷 보내야 하므로)
            _movePacketSendTimer = 0.f;
        }
    }
}
```



1. Space Robber



게임 구현 설명

▼ 서버에서 수신한 패킷 게임 월드에 반영

- GameNetwork 클래스에서 Recv Thread로 패킷을 수신하여 오브젝트에 반영했습니다.
- 게임 월드 내의 플레이어, 몬스터, 아이템, 건물과 문 등은 GameInstance 에서 고유 ID와 객체 포인터로 매핑되어 관리됩니다.

```
// 다른 플레이어
UPROPERTY(EditAnywhere)
1개의 Blueprint 에서 변경됨
TSubclassOf<AOtherPlayer> OtherPlayerClass;

TMap<uint64, AOtherPlayer*> _otherPlayers; // ID, Character*

// 내 플레이어
AMyPlayer* _myPlayer;
int _myID(-1);
int _clientId(-1);

// 몬스터
UPROPERTY()
0 Blueprints에서 변경됨
TMap<uint64, ABaseMonster*> _monsters;

// 아이템
UPROPERTY()
0 Blueprints에서 변경됨
TMap<uint64, ABaseItem*> _items;
// 장비
UPROPERTY()
0 Blueprints에서 변경됨
TMap<uint64, ABaseItem*> _tools;

// 큐브
UPROPERTY()
0 Blueprints에서 변경됨
TArray<AStaticMeshActor*> _cubes;
```

• 로그인 처리

```
void RecvSignupResult(S_SignupResult_Packet packet);
void RecvLoginResult(S_LoginResult_Packet packet);
void RecvCurrentRoomList(S_CurrentRoomList_Packet packet);
void RecvCreateRoomResultList(S_CreateRoomResult_Packet packet);
void RecvEnterRoomResultList(S_EnterRoomResult_Packet packet);
```

• 객체 동기화

```
void RecvAddPlayer(S_AddPlayer_Packet packet);
void RecvAddItem(S_AddItem_Packet packet);
void RecvAddTool(S_AddItem_Packet packet);
void RecvDropItem(S_DropItem_Packet packet);

void RecvRemoveObject(S_RemoveObject_Packet packet);
void RemovePlayer(S_RemoveObject_Packet packet);
void RemoveMonster(S_RemoveObject_Packet packet);
void RemoveItem(S_RemoveObject_Packet packet);
void RemoveObstacle(S_RemoveObject_Packet packet);
void RemoveDoor(S_RemoveObject_Packet packet);
void RemoveSellingMachine(S_RemoveObject_Packet packet);

void RecvMoveObject(S_Move_Packet packet);
void RecvMovePlayer(S_Move_Packet packet);
void RecvMoveMonster(S_Move_Packet packet);

void RecvUpdateObjectState(S_UpdateObjectState_Packet packet);
void RecvUpdateStateMonster(S_UpdateObjectState_Packet packet);
void RecvUpdateStateDoor(S_UpdateObjectState_Packet packet);
void RecvUpdateStateSellingMachine(S_UpdateObjectState_Packet packet);
void RecvUpdateStatePlayer(S_UpdateObjectState_Packet packet);
```

• 퀘스트, 마이크

```
void RecvUpdateQuest(S_UpdateQuest_Packet packet);
void RecvUpdateQuestProgress(S_UpdateQuestProgress_Packet packet);
void RecvUpdateCredit(S_UpdateCredit_Packet packet);
void RecvUpdateTimer(S_UpdateTimer_Packet packet);

void RecvVoiceData(S_VoiceData_Packet packet);
```



1. Space Robber



게임 구현 설명

▼ OpenCV 기반 실시간 감정 인식 시스템 구현

- OpenCV 라이브러리를 Unreal Engine 5.6 프로젝트에 직접 연동하여 웹캠 영상을 실시간으로 입력 받고, 플레이어의 감정 상태를 분석하는 기능을 구현했습니다.
- [OpenCV Haar Cascade](#)를 이용해 웹캠 영상에서 얼굴을 검출한 후, [FER2013](#) 데이터셋으로 학습된 [ONNX](#) 기반 감정 분류 모델을 추론 엔진에 연동하여 Happy, Sad, Angry, Surprise, Fear, Disgust, Neutral의 7가지 감정을 실시간으로 분류했습니다.
- 실시간 감정 인식 결과는 프레임마다 미세한 확률 변화가 발생하므로 1초 주기로 감정을 추출하도록 구현했습니다. 추출된 7개의 감정 확률 중 가장 높은 값을 현재 감정으로 선택하고, 이전에 전송한 감정 상태와 비교하여 변화가 발생한 경우에만 서버로 감정 패킷을 전송했습니다.

- 이를 통해 불필요한 네트워크 패킷을 최소화하면서도 플레이어의 감정 변화를 실시간으로 게임 시스템에 반영할 수 있도록 구현했습니다.

```
// 전처리
cv::cvtColor(frame, gray, cv::COLOR_BGR2GRAY);
cv::equalizeHist(gray, gray);

std::vector<cv::Rect> faces;
_faceDetector.detectMultiScale(gray, faces, 1.1, 5, 0, cv::Size(30, 30));

if (faces.size() > 0)
{
    // 가장 큰 얼굴 선택
    faceROI = gray(faces[0]);

    // 모델 입력 규격 맞추기 (48x48, 1/255 정규화)
    cv::Mat blob = cv::dnn::blobFromImage(faceROI, 1.0 / 255.0, cv::Size(48, 48), cv::Scalar(0, false, false));

    _emotionNet.setInput(blob);
    cv::Mat output = _emotionNet.forward();

    // LogSoftmax를 확률로 변환
    cv::Mat prob;
    cv::exp(output, prob);

    // 현재 초의 감정 확률을 배열에 담음
    TArray<float> currentScores;
    currentScores.Init(0.f, 7);
    for (int i = 0; i < 7; ++i)
    {
        currentScores[i] = prob.at<float>(0, i);
    }

    // Main에 데이터 전달(서버에 전송 및 UI 그리기는 Main에서..)
    if (_main)
    {
        AsyncTask(ENamedThreads::GameThread, [_main = this->_main, currentScores]()
        {
            if (_main)
                _main->HandleNewEmotionData(currentScores);
        });
    }
}

FPlatformProcess::Sleep(1.0f);
}
```

• 감정 추출 로직

```
LogTemp: Warning: >> Result: Happy (67.6%)
LogTemp: Warning: >> Result: Happy (61.6%)
LogTemp: Warning: >> Result: Neutral (29.8%)
LogTemp: Warning: >> Result: Sad (36.0%)
LogTemp: Warning: >> Result: Neutral (39.5%)
LogTemp: Warning: >> Result: Fear (43.6%)
LogTemp: Warning: >> Result: Fear (41.3%)
LogTemp: Warning: >> Result: Happy (86.4%)
LogTemp: Warning: >> Result: Happy (30.7%)
LogTemp: Warning: >> Result: Happy (30.7%)
```

• 감정 데이터 로그

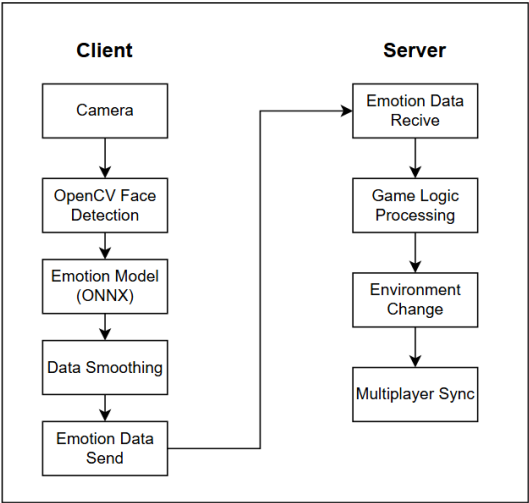
1. Space Robber

게임 구현 설명

▼ 감정 데이터 서버로 송신

- 추출된 감정 데이터를 UE 내부 게임 시스템과 연동하여 서버로 전송하였으며, 향후 게임 시스템에 활용할 수 있도록 설계했습니다.
- 감정 상태를 반영한 몬스터 AI 행동 변화
- 플레이어 감정에 따른 동적 난이도 조절

· 서버-클라이언트 흐름도



· 몬스터와의 상호작용



▼ 한국게임학회 학술대회

- OpenCV 기반 감정 인식 기능을 게임 환경에 적용한 연구 결과를 바탕으로 「OpenCV 기반 실시간 감정 인식을 활용한 몰입형 게임 환경 설계」 논문을 작성하여 한국게임학회 학술대회에서 포스터 발표를 진행했습니다. 연구에서는 감정 인식 기술을 게임 플레이와 연계하는 시스템 구조와 적용 방법을 발표하였습니다.
- 연구를 통해 게임 분야에서 AI 기반 감정 분석 기술의 활용성을 검증했습니다.

[논문 보러가기](#)



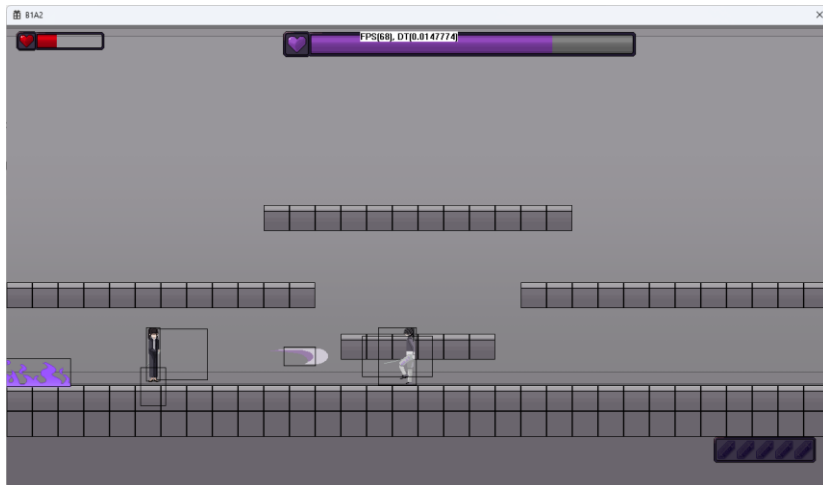
2. Magical Subway



프로젝트

▼ 게임 설명

- WinAPI 2D 횡스크롤 액션 어드벤처 게임(C++)
- 액션 어드벤처 장르를 기반으로 제작한 싱글 플레이 게임으로, 플레이어는 다양한 구조물과 상호작용하며 아이템을 수집하고 퍼즐을 해결해 나갑니다. 맵에 배치된 몬스터를 처치하며 최종 보스를 물리치는 것을 목표로 하는 게임입니다.



▼ 게임 개발

- 인원: 3명(클라이언트 2명 – 본인, 기획 1명)
- 기간: 2024.09 ~ 2025.07 (11개월)
- 담당 업무
 - 클라이언트(FSM · Behavior Tree 시스템, 아이템 · 인벤토리 · UI, 사운드 시스템, 퍼즐)
- 목적: 게임 클라이언트를 직접 구현하며 게임 엔진의 핵심 구조와 렌더링, 입력 처리, 게임 루프 등 게임 실행 원리를 이해하고 구현하는 것을 목표로 했습니다.
- 기술: C++, WinAPI



2. Magical Subway



게임 구현 설명

▼ WinAPI를 사용한 클라이언트 개발

- 상용 게임 엔진을 사용하지 않고 WinAPI를 기반으로 게임 루프, 입력 처리, 렌더링, 오브젝트 관리, 리소스 관리, 행동 트리 처리 등 게임 실행의 핵심 구조를 직접 구현했습니다. 이를 통해 게임 엔진의 내부 동작 방식과 클라이언트 핵심 구조를 이해했습니다.

▼ CSV 기반 파일 입출력 시스템

- CSV 파일을 활용하여 플레이어의 진행 상황을 저장하고 불러오는 기능을 구현하여 게임을 이어서 플레이할 수 있도록 했습니다.
- 플레이어, 몬스터, 아이템, 구조물 등의 스탯 데이터를 CSV에서 load 하도록 설계하여 데이터 수정 시 코드 변경 없이 기획 데이터만으로 조정할 수 있도록 데이터 구조를 정립했습니다.

▼ 클래스와 상속 구조 설계

- 플레이어, 몬스터, 구조물, 투사체 등 게임 오브젝트의 공통 기능을 기반 클래스로 정의하고 상속 구조를 설계했습니다. FSM을 기반으로 각 객체의 상태를 관리했으며, 스탯, 충돌 박스, 애니메이션(플립북) 등의 기능을 클래스 단위로 분리하여 유지보수성과 확장성을 높였습니다.

▼ Behavior Tree 기반 AI 시스템 구현

- 몬스터 AI를 위해 Behavior Tree를 직접 구현했습니다. Selector, Sequence, Action, Condition 노드를 설계하여 행동 트리를 구성하고, 각 노드가 SUCCESS, FAIL, RUNNING 상태를 반환하도록 구현했습니다. 이를 통해 조건에 따른 행동 선택과 우선순위 기반의 AI 로직을 유연하게 구성하도록 했습니다.



2. Magical Subway



게임 구현 설명

▼ 인벤토리 시스템

- 아이템 정보를 CSV에서 읽어와 인벤토리에 등록하고, 선택한 아이템의 아이콘, 설명, 보유 개수를 UI에 출력하는 기능을 구현했습니다.

▼ FMOD를 활용한 사운드 시스템 구현

- 사운드 매니저를 구현하여 FMOD 라이브러리를 연동하였습니다. BGM과 효과음을 관리하고, 플레이어와 몬스터의 행동이나 게임 상황에 따라 적절한 사운드가 재생되도록 구현하였습니다.

▼ 콘텐츠 및 기믹 시스템 구현

- 짚라인, 이동 발판, 장애물 등 다양한 기믹 오브젝트와 플레이어 간의 상호작용 시스템을 구현했습니다. 각 오브젝트의 상태와 동작을 독립적으로 관리하여 이동 및 퍼즐 요소를 구성하고, 레벨 디자인에 맞는 콘텐츠를 개발했습니다.



3. 대초원의 모험

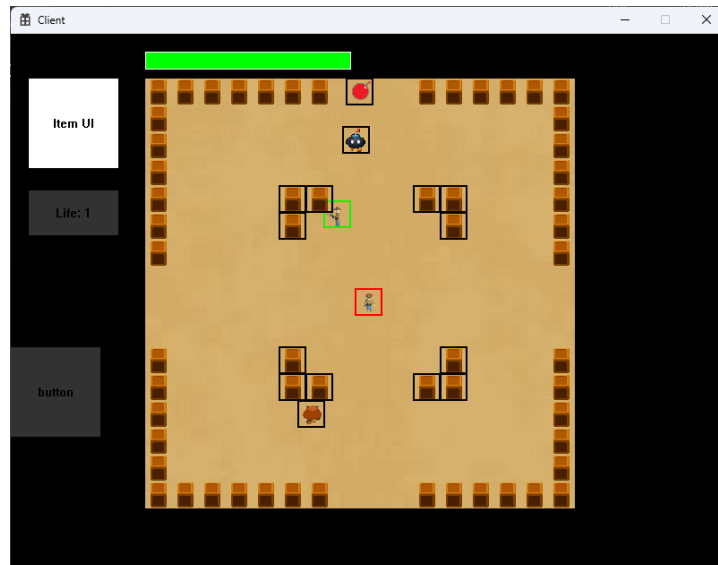


네트워크 게임 프로그래밍 최종 프로젝트



▼ 게임 설명

- WinAPI 2D 횡스크롤 액션 어드벤처 게임(C++)
- 액션 어드벤처 장르를 기반으로 제작한 싱글 플레이 게임으로, 플레이어는 다양한 구조물과 상호작용하며 아이템을 수집하고 퍼즐을 해결해 나갑니다. 맵에 배치된 몬스터를 처치하며 최종 보스를 물리치는 것을 목표로 하는 게임입니다.



▼ 게임 개발

- 인원: 3명(클라이언트 2명 – 본인, 서버 1명)
- 기간: 2025.10 ~ 2025.12
- 담당 업무
 - 클라이언트 네트워크 시스템 개발
 - 팀장 및 프로젝트 일정 관리
 - 설계 문서 및 진행 보고서 작성 관리
- 목적: WinAPI와 Winsock 기반의 멀티플레이 게임 클라이언트를 개발하며, 서버와의 네트워크 통신 및 객체 동기화 시스템을 구현하고 실시간 멀티플레이 게임의 동작 원리를 이해하는 것을 목표로 했습니다.
- 기술: C++, WinAPI, Winsock(TCP/IP), Select I/O Model, Multi Thread

감사합니다.

Contact

 010-5744-2840  ce0118ce@naver.com